

Travaux Pratiques : Détection d'Intrusion avec Snort IDS

BTS SIO - Option SISR

Étudiant : Louis DENIEL

Date : Mars 2026

Contexte : Sécurité des Systèmes d'Information

Objectifs du TP

Ce travail pratique vise à maîtriser l'utilisation de Snort, un système de détection d'intrusion réseau (IDS - Intrusion Detection System) dans un environnement Linux. Les objectifs pédagogiques sont les suivants :

- Comprendre le fonctionnement d'un IDS réseau et son rôle dans la sécurité
- Configurer et personnaliser des règles Snort pour détecter des comportements suspects
- Analyser le trafic réseau mis en miroir (port mirroring) avec iptables
- Identifier les limites des IDS face au chiffrement SSL/TLS •
Différencier la surveillance du trafic interne et externe

Environnement Technique

Infrastructure Labtainer

Le laboratoire utilise l'environnement Labtainer, développé par le Naval Postgraduate School, qui permet de simuler des infrastructures réseau complexes via des conteneurs Docker.

Commandes de démarrage :

```
export DISPLAY=:0
```

```
labtainer snort
```

Architecture Réseau

L'infrastructure simulée comprend plusieurs composants interconnectés :

Composant	Adresse IP	Rôle
-----------	------------	------

Passerelle (Gateway)	203.0.113.10 (externe) 192.168.1.10 (interne)	NAT, redirection, port mirroring
Serveur Web	192.168.1.2	Apache avec SSL (www.example.com)
Snort IDS	192.168.3.1	Détection d'intrusion, Wireshark
Poste distant (remote_ws)	203.0.113.x	Simulation attaquant externe
Poste interne (ws2 - Mary)	192.168.2.1	Utilisateur interne

Table 1: Composants de l'infrastructure réseau

Configuration iptables

La passerelle joue un rôle central avec plusieurs fonctions critiques :

1. Translation NAT : Les adresses internes (192.168.x.x) sont traduites vers l'adresse externe (203.0.113.10)
2. Redirection de ports : Le trafic web (ports 80 et 443) est redirigé vers le serveur web interne
3. Port Mirroring : Le trafic est dupliqué et envoyé vers le composant Snort pour analyse. Cette configuration est définie dans /etc/rc.local :

Mise en miroir du trafic externe

```
iptables -t mangle -A PREROUTING -i $wan -j TEE --gateway
192.168.3.1
```

Mise en miroir du trafic du serveur web

```
iptables -t mangle -A PREROUTING -i $lan1 -j TEE --gateway
192.168.3.1
```

Cette technique permet à Snort de reconstruire les sessions TCP complètes et d'analyser le trafic sans interférer avec les communications légitimes.

Travaux Réalisés

Tâche 1 : Démarrage et Arrêt de Snort

Objectif : Maîtriser les commandes de base pour contrôler Snort

Procédure :

1. Connexion au terminal du composant Snort
2. Lancement de Snort en mode IDS avec le script fourni :
`./start_snort.sh`
3. Observation des alertes en temps réel sur la console
4. Arrêt de Snort avec CTRL+C pour modification des règles

Résultat : Snort démarre en mode détection d'intrusion et affiche les alertes directement sur la console.

Tâche 2 : Règles Snort Préconfigurées

Objectif : Identifier les règles de détection par défaut

Procédure :

1. Depuis le poste distant (remote_ws), exécution d'un scan nmap :
`sudo nmap www.example.com`
2. Observation des alertes générées par Snort
3. Analyse des règles préconfigurées dans `/etc/snort/rules/`

Résultats observés :

Les règles par défaut de Snort ont détecté plusieurs comportements suspects liés au scan nmap, notamment :

- Détection de paquets ICMP PING caractéristiques de nmap
- Identification de tentatives de scan de ports
- Alertes sur des patterns de reconnaissance réseau

Analyse : Les règles préconfigurées de Snort couvrent de nombreux comportements malveillants connus, issus de la base de données communautaire. Ces règles sont régulièrement mises à jour pour détecter les nouvelles menaces.

Tâche 3 : Création d'une Règle Simple (Test)

Objectif : Comprendre la syntaxe des règles Snort et leurs limites

Règle créée :

```
alert tcp any any -> any any (msg:"TCP detected"; sid:00002;)
```

Syntaxe expliquée :

alert : Action à effectuer (générer une alerte)

tcp : Protocole concerné

any any : N'importe quelle adresse source et n'importe quel port source

-> : Direction du trafic

any any : N'importe quelle adresse destination et n'importe quel port destination

msg:"TCP detected" : Message affiché lors de l'alerte

sid:00002 : Identifiant unique de la règle

Test réalisé :

1. Redémarrage de Snort avec la nouvelle règle
2. Lancement de Firefox depuis le poste distant :
firefox www.example.com
3. Observation des résultats

Résultat : Submersion d'alertes ! Chaque paquet TCP génère une alerte, rendant l'information complètement inutile.

Leçon apprise : Une règle trop générique crée du bruit et masque les véritables menaces. Les règles doivent être spécifiques et ciblées pour être efficaces. Cette règle a été supprimée immédiatement.

Tâche 4 : Règle de Détection de Contenu Confidentiel

Objectif : Détecter l'accès à des informations sensibles par analyse du contenu

Contexte : L'entreprise héberge des plans confidentiels à l'adresse www.example.com/plan.html. Nous devons être alertés lorsque ce contenu contenant le mot "CONFIDENTIAL" transite sur le réseau.

Règle développée :

```
alert tcp any any -> any any (msg:"CONFIDENTIAL detected";  
content:"CONFIDENTIAL"; nocase; sid:10000002;)
```

Composants de la règle :

content:"CONFIDENTIAL" : Détecte le texte spécifié dans la payload du paquet
nocase : Rend la détection insensible à la casse (confidential, CONFIDENTIAL, Confidential)
sid:10000002 : Identifiant unique pour éviter les conflits avec les règles existantes

Procédure de test :

1. Ajout de la règle dans /etc/snort/rules/local.rules
2. Redémarrage de Snort
3. Sur le poste distant, effacement de l'historique Firefox (Menu → Préférences → Sécurité et confidentialité)
4. Accès à la page : <http://www.example.com/plan.html>
5. Observation des alertes Snort

Résultat : ☒ Alerte générée avec succès

Snort détecte le mot "CONFIDENTIAL" dans la réponse HTTP du serveur web et génère une alerte contenant :

L'adresse IP source (serveur web)
L'adresse IP destination (poste distant)
Le message personnalisé "CONFIDENTIAL detected"
L'horodatage de la détection

Tâche 5 : Impact du Chiffrement SSL/TLS

Objectif : Comprendre les limitations des IDS face au trafic chiffré

Procédure :

1. Effacement de l'historique Firefox
2. Modification de l'URL pour utiliser HTTPS :
<https://www.example.com/plan.html>
3. Observation de la console Snort

Résultat : ✗ Aucune alerte générée

Analyse technique :

Le protocole HTTPS chiffre l'intégralité de la communication HTTP, incluant :

L'URL de la requête (y compris /plan.html)
Les en-têtes HTTP
Le corps de la réponse (contenant "CONFIDENTIAL")

Snort ne peut analyser que les données en clair. Lorsque le trafic est chiffré avec SSL/TLS, l'IDS voit uniquement :

- L'établissement de la connexion TLS (handshake)
- Les paquets chiffrés indéchiffrables
- Les métadonnées (adresses IP, ports, taille des paquets)

Solution théorique proposée :

Architecture avec Reverse Proxy

1. Déployer un reverse proxy devant le serveur web
2. Le proxy gère le chiffrement SSL/TLS avec les clients
3. La communication entre le proxy et le serveur web reste en HTTP clair
4. Le trafic HTTP non chiffré peut être reflété vers Snort
5. L'IDS peut alors analyser le contenu en clair

Avantages :

- Maintien de la sécurité SSL/TLS pour les utilisateurs externes
- Détection d'intrusion efficace sur le trafic déchiffré
- Point de terminaison SSL centralisé

Inconvénients :

- Complexité architecturale accrue
- Nécessite des ressources supplémentaires
- Point unique de défaillance potentiel

Tâche 6 : Surveillance du Trafic Interne

Objectif : Étendre la détection au réseau interne

Test initial :

1. Depuis le poste ws2 (Mary), exécution de nmap :
sudo nmap www.example.com
2. Observation de la console Snort

Résultat initial : Alertes partielles uniquement

Snort détecte uniquement les réponses provenant du serveur web (192.168.1.2), mais pas les requêtes provenant du poste de Mary (192.168.2.1).

Explication : Le trafic du réseau interne de Mary (LAN2) n'était pas initialement reflété vers Snort. Seul le trafic externe et celui du serveur web étaient mis en miroir.

Solution implémentée :

Modification du fichier `/etc/rc.local` sur la passerelle pour ajouter la règle suivante :

```
iptables -t mangle -A PREROUTING -i $lan2 -j TEE --gateway  
192.168.3.1
```

Cette commande duplique tout le trafic entrant sur l'interface `$lan2` (réseau de Mary) et l'envoie vers Snort (192.168.3.1).

Application de la configuration :

```
sudo /etc/rc.local
```

Test après modification :

1. Redémarrage de Snort
2. Nouvelle exécution de nmap depuis ws2
3. Observation des alertes

Résultat : ☒ Détection complète

Snort génère désormais les alertes "ICMP PING NMAP" pour les paquets provenant du poste de Mary, permettant une surveillance complète du trafic interne.

Tâche 7 : Différenciation Trafic Interne/Externe

Objectif : Créer des règles intelligentes permettant aux utilisateurs internes d'accéder aux ressources confidentielles sans générer d'alertes, tout en surveillant les accès externes.

Problématique : Les employés doivent pouvoir consulter leur plan d'affaires confidentiel sans déclencher d'alertes, mais tout accès externe doit être détecté.

Contraintes à respecter :

1. L'accès externe au plan d'affaires doit générer une alerte
2. L'accès interne au plan d'affaires ne doit PAS générer d'alerte
3. Les scans nmap (internes ou externes) doivent générer une alerte ICMP NMAP PING

Analyse du flux réseau avec NAT :

Point clé à comprendre : En raison de la NAT configurée sur la passerelle, le comportement du trafic diffère selon l'origine :

Type d'accès	Adresse source	Adresse destination
Externe → Serveur web	203.0.113.x	192.168.1.2
Serveur → Client externe	192.168.1.2	192.168.1.10 (passerelle)
Serveur → Client interne	192.168.1.2	192.168.2.1 (Mary)

Table 2: Flux réseau selon l'origine

Règle optimisée développée :

alert tcp 192.168.1.2 80 -> 192.168.1.10 any (msg:"CONFIDENTIAL detected"; content:"CONFIDENTIAL"; nocase; sid:10000003;)

Explication de la règle :

192.168.1.2 : Source = Serveur web uniquement

80 : Port source = HTTP (port du serveur web)

192.168.1.10 : Destination = Passerelle (trafic NAT vers l'extérieur)

any : N'importe quel port destination

Cette règle cible spécifiquement le trafic du serveur web destiné à sortir vers l'extérieur

Logique de filtrage :

- - ☒ Trafic externe : Serveur web (192.168.1.2) → Passerelle (192.168.1.10) → Internet ⇒ Alerte générée
- - ☒ Trafic interne : Serveur web (192.168.1.2) → Mary (192.168.2.1) ⇒ Aucune alerte (destination différente) •
 - ☒ Scan nmap : Alertes ICMP existantes fonctionnent indépendamment

Tests de validation réalisés :

Test 1 - Accès interne :

1. Depuis ws2 (Mary), effacement de l'historique Firefox
2. Accès à <http://www.example.com/plan.html>

3. Résultat : ☒ Aucune alerte générée (comportement attendu)

Test 2 - Accès externe :

1. Depuis remote_ws, effacement de l'historique Firefox 2.
Accès à <http://www.example.com/plan.html>

3. Résultat : ☒ Alerte "CONFIDENTIAL detected" générée

Test 3 - Scan nmap externe :

1. Depuis remote_ws : `sudo nmap www.example.com` 2.

Résultat : ☒ Alertes ICMP NMAP PING générées

Test 4 - Scan nmap interne :

1. Depuis ws2 (Mary) : `sudo nmap www.example.com` 2.

Résultat : ☒ Alertes ICMP NMAP PING générées

Validation finale : Tous les critères sont respectés simultanément lors d'une seule session Snort.

Validation et Soumission

Vérification Automatique

Le système Labtainer intègre un mécanisme de validation automatique :

`checkwork`

Cette commande vérifie que les objectifs pédagogiques ont été atteints selon les critères définis par les formateurs.

Arrêt du Laboratoire

`stoplab snort`

Le système génère un fichier ZIP contenant :

- Les fichiers de configuration modifiés

- Les règles Snort personnalisées créées

- Les logs des tests effectués

- Les résultats de la validation automatique

Évaluation Formateur

Le formateur peut noter le travail avec :

```
cd ../labtainer-instructor
gradelab snort
```

Compétences Développées

Compétences Techniques

- Configuration IDS : Installation, paramétrage et optimisation de Snort
- Analyse réseau : Compréhension approfondie du port mirroring et de la capture de paquets
- Règles de détection : Création de signatures personnalisées avec syntaxe Snort
- Troubleshooting : Diagnostic des problèmes de détection et ajustement des règles
- Sécurité réseau : Compréhension des limites du chiffrement pour les IDS
- NAT/iptables : Maîtrise de la translation d'adresses et du routage avancé

Compétences Transversales

- Analyse critique : Évaluation des forces et faiblesses des solutions de sécurité
- Documentation technique : Production de documentation professionnelle
- Résolution de problèmes : Méthodologie structurée face aux incidents
- Veille technologique : Compréhension des enjeux actuels (chiffrement vs détection)

Réflexions et Améliorations

Points Forts de la Solution

1. Détection granulaire : Différenciation efficace entre trafic interne et externe
2. Règles ciblées : Minimisation des faux positifs grâce à des règles

- spécifiques
- 3. Architecture évolutive : Possibilité d'ajouter facilement de nouvelles règles
- 4. Surveillance complète : Couverture du trafic interne et externe

Limites Identifiées

1. Chiffrement SSL/TLS : Impossibilité d'analyser le contenu chiffré sans déchiffrement
2. Performance : Impact potentiel sur les performances avec des règles complexes
3. Maintenance : Nécessité de mettre à jour régulièrement les règles
4. Faux négatifs : Risque de ne pas détecter des attaques sophistiquées ou inconnues

Perspectives d'Amélioration

1. Déploiement d'un reverse proxy avec terminaison SSL pour permettre l'inspection du trafic HTTPS
2. Intégration avec un SIEM (Security Information and Event Management) pour corrélation des événements
3. Automatisation : Scripts de mise à jour automatique des règles depuis les bases communautaires
4. Machine Learning : Utilisation d'algorithmes d'apprentissage pour détecter les comportements anormaux
5. Haute disponibilité : Configuration de Snort en cluster pour redondance

Conclusion

Ce travail pratique a permis de développer une expertise opérationnelle sur Snort IDS, outil de référence en détection d'intrusion réseau. Les manipulations réalisées ont démontré :

L'efficacité des IDS pour détecter des comportements suspects et des contenus sensibles

L'importance de la configuration réseau (port mirroring) pour une surveillance efficace

Les limites inhérentes face au chiffrement moderne (SSL/TLS) La nécessité de créer des règles de détection intelligentes et contextualisées

Application professionnelle : Dans le contexte de mon alternance chez Volvo CE, ces compétences sont directement transposables pour la sécurisation de l'infrastructure réseau de l'entreprise. La mise en place

d'un IDS comme Snort permettrait de détecter les tentatives d'intrusion, les scans de reconnaissance et les fuites de données sensibles.

Perspective BTS SIO : Ce TP s'inscrit parfaitement dans le référentiel SISR, couvrant les compétences liées à la cybersécurité, la gestion des infrastructures réseau et la mise en œuvre de solutions de supervision. La documentation produite pourra être réutilisée dans le cadre des épreuves professionnelles.

Annexes

Ressources Consultées

1. Manuel utilisateur officiel Snort : <https://www.snort.org/documents/snort-users-manual>
2. Port mirroring avec iptables : <https://www.it-connect.fr/port-mirroring-sous-linux-avec-iptables/>
3. Limitations des IDS face à HTTPS : <https://security.stackexchange.com/a/216475>
4. CNIL - Recommandations sécurité : ressources complémentaires sur la protection des données

Commandes Essentielles

Commande	Description
<code>./start_snort.sh</code>	Démarrer Snort en mode IDS
<code>CTRL+C</code>	Arrêter Snort
<code>sudo nmap <target></code>	Scanner un hôte réseau
<code>iptables -t mangle -L</code>	Lister les règles de port mirroring
<code>sudo /etc/rc.local</code>	Appliquer les règles iptables
<code>checkwork</code>	Vérifier la progression du TP
<code>stoplab snort</code>	Arrêter le laboratoire et générer le rapport

Table 3: Commandes utilisées durant le TP

Fichiers de Configuration Clés

`/etc/snort/rules/local.rules` : Règles personnalisées
`/etc/snort/rules/` : Répertoire des règles préconfigurées
`/etc/rc.local` : Configuration iptables pour port mirroring
`/etc/hosts` : Mapping DNS local sur les postes de travail

Schéma d'Architecture (Référence)

Voir Figure 1 du document source pour la topologie réseau complète incluant les adresses IP, les interfaces réseau et les flux de données.